Part A　Software Engineering

第 1 回　ソフトウェア工学概説

1. Agent-Oriented Software Engineering [22.2] **(4)** [21.8] **(4)**

Explain the **agent-oriented software engineering**.

Agent-oriented software engineering (AOSE) is a method for developing software as a group of autonomous agents. Each agent can observe its environment, make decisions, communicate with other agents, and act to achieve a goal. For example, in an online shopping system, buyer agents, seller agents, and delivery agents cooperate to complete an order.

AOSE is useful for complex and distributed systems because it provides autonomy, flexibility, and scalability.

第 2 回　ソフトウェアプロセス

1. Waterfall Model 与 Agile Model 的比较 [25.2] **(1)**[23.8] **(1)**[23.2] **(1)**[22.8] **(1)**[22.2] **(1)**[21.8] **(1)**

About software developing processes, explain the **waterfall development model** and the **agile development model** with their advantages and disadvantages.

①Waterfall is a sequential development model. Each phase is completed before moving to the next phase.

Advantages: easy to manage，clear documentation

Disadvantages: inflexibility and costly rework

②Agile is an iterative and incremental development model that delivers software in small iterations.

Advantages: flexible, customer feedback

Disadvantages: less predictable, customer involvement

->waterfall is suitable for stable requirements, whereas Agile is suitable for changing requirement.

---

2. Waterfall Lifecycle 的阶段

五个阶段 [26.2] **(1-a)** [24.2] **(1-a)**

Divide the waterfall life cycle into five important phases (**requirements definition, external design, internal design, implementation, and testing**) and explain each phase.

① requirements definition

This phase determines what software should be developed by analyzing user needs.

It produces a requirements specification.

② external design

This phase designs what the system should do from the user's viewpoint, including functions, inputs, outputs, and interfaces.

It produces an external design specification.

③ internal design

This phase designs how the system will be implemented by dividing it into modules and defining their relationships.

It produces an internal design specification.

④ implementation

.This phase writes the program module by module according to the design.

It produces the source code.

⑤ Testing

This phase checks whether each module and the whole system work correctly and satisfy the requirements.

It produces a test report.

四个阶段 [24.8] **(1)**

The main 4 steps of traditional software development processes are classified into the **requirement analysis step, the design step, the implementation step, and the testing step**. Explain these 4 steps.

① requirements analysis

② design

③ implementation

④ testing

3. Incremental Development 与 Iterative Development [26.2] **(1-c)** [24.8] **(2)**

On the software development process, explain the difference between the **incremental development process** and the **iterative development process**.

① Incremental development builds a system by adding new functional components in a series of increments. Each increment provides additional functionality and expands the system.

② Iterative development builds a system through repeated cycles of refinement. Each iteration improves the existing system base on evaluation and feedback.

---

4. Waterfall Model 中的 Rework [26.2] **(1-b)** [24.2] **(1-b)**

Discuss "rework" in the waterfall model from the perspective of its causes, problems, and prevention methods.

① **Causes:** requirements unclear, requirements change, design mistakes, poor communication

② **Problems:** increase development cost, delays project schedules, reduce productivity, lower quality, customer dissatisfaction

③ **Prevention methods:** through requirements analysis, customer involvement, design reviews, prototyping, verification and validation

---

第 3 回　要求工学（1）

1. Prototyping 的概念与重要性 [26.2] **(2-a)** [25.8] **(1)** [25.2] **(2)** [24.2] **(2)**

Explain what **prototyping** is and discuss its importance.

① Prototyping is a process of creating a preliminary version of a software system to clarify the requirements before developing the final product.

② Importance: clarifies requirements, impress communication, obtains early feedback, reduces rework, reduces risk, improves user satisfaction.

---

2. Feasibility Study [23.8] **(3)**

Explain what is **"Feasibility study"** and discuss its importance.

A **feasibility study** is the process of evaluating whether a software project is **practical and worth developing** before starting the development.

It examines factors such as:

- **Technical feasibility**: Can the required technology be developed?
- **Economic feasibility**: Are the cost and benefits reasonable?
- **Schedule feasibility**: Can the project be completed within the required time?

The importance of a feasibility study is that it helps organizations **identify risks, reduce unnecessary costs, and decide whether to proceed with the project**.

第 4 回　モデル化技法と UML（1）

1. Use Case Diagram 的作用 [23.2] **(2)** [22.8] **(2)** [22.2] **(2)** [21.8] **(2)**

Explain the role of the **use case diagram** in UML with an example.

A **use case diagram** shows the system's functions and the interactions between the system and external actors.

It is used to clarify **who uses the system and what they can do**, rather than how the system is internally implemented.

For example, in an **ATM system**:

- A **customer** can withdraw money, transfer money, and deposit money.
- A **system manager** can register ATMs and read logs.
- A **bank** processes deposits and ATM registration.

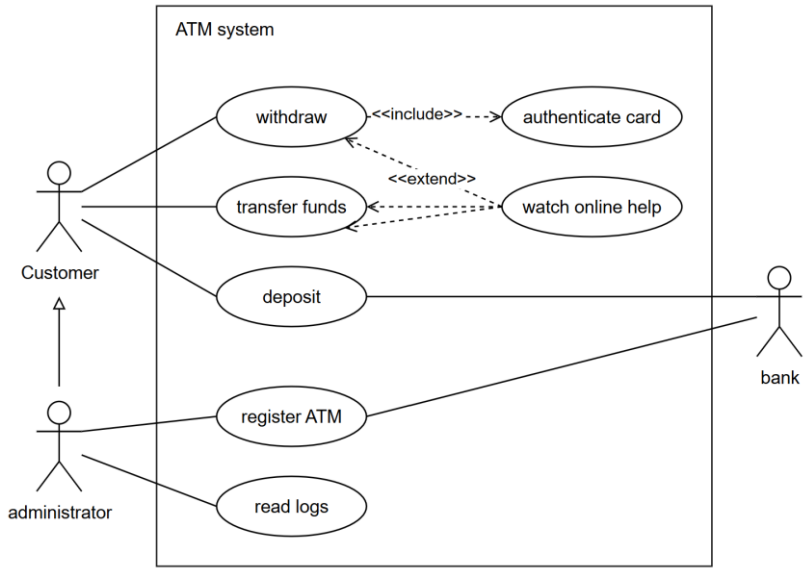
Therefore, the use case diagram helps developers and users understand and confirm the system requirements.

---

2. ATM Banking System 用例图 [26.2] **(3-a)** [25.8] **(2)** [24.8] **(4)** [23.8] **(4)**

Draw a simple use case diagram about **banking ATMs (Automatic Teller Machines)**.Create at least **three distinct**

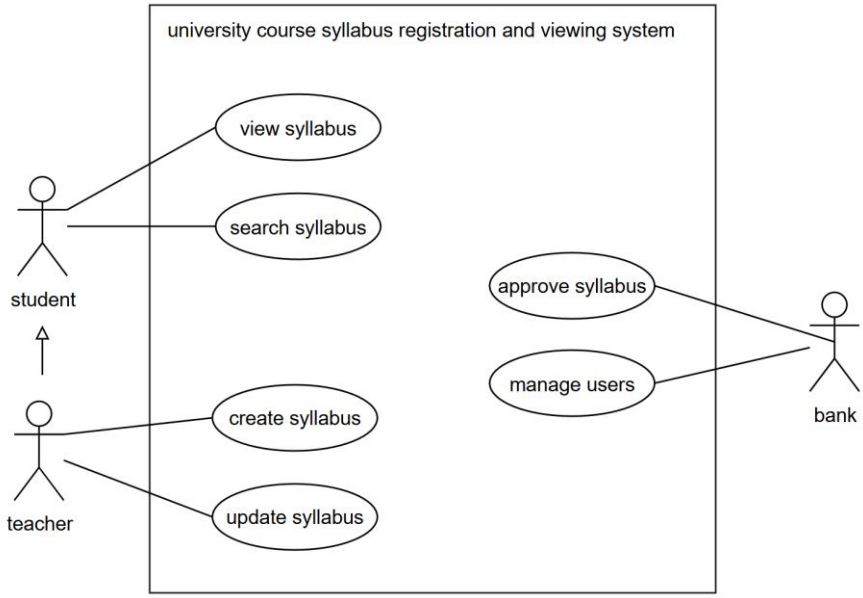
actors and associate each actor with at least **two use cases**.



---

3. University Syllabus Management System 用例图 [26.2] **(3-b)** [25.2] **(3)**

Draw a simple use case diagram for the **university course syllabus registration and viewing system**. Create at least **three distinct actors** and associate each actor with at least **two use cases**.

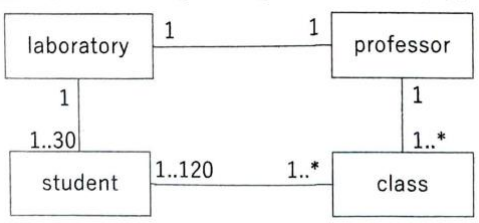


第 5 回　モデル化技法と UML（2）

1. Class Diagram 中的 Multiplicity [24.8] **(5)**

Explain the following **class diagram** in terms of **multiplicity**.

(Refer to the given class diagram and explain the meaning of each multiplicity relationship.)



Laboratory-professor: one laboratory can only belong to one professor, professor can only have one laboratory.
Laboratory-student: one laboratory can hold one to thirty students, every student can only belong to one laboratory.
Student-class: every student can take 1 to many classes, every class can hold 1 to 120 students.
Professor-class: one professor can teach 1 to many classes, one class can only be taught by one professor.

---

2. Automobile Class Diagram 中的关系

Association [24.2] **(5-a)**

Provide the name of the relationship between **"Owner"** and **"Automobile"**. Also, explain the meaning of the relationship.

Association: one owner can have 0 or many automobiles and each automobile has exactly one owner.

Generalization [24.2] **(5-b)**

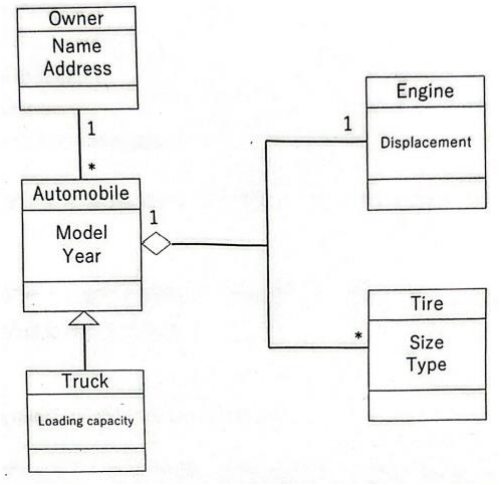
Provide the name of the relationship between **"Automobile"** and **"Truck"**. Also, explain the meaning of the relationship.

Generalization: A truck is a type of automobile and inherits the attributes and operations of Automobile.

Aggregation [24.2]**(5-c)**

Provide the name of the relationship between **"Automobile," "Engine," and "Tire."** Also, explain the meaning of the relationship.

Aggregation: An engine and tires are components of an automobile. Each automobile has exactly one engine and zero or more tires. These components can exist independently of the automobile.



第 6 回　オブジェクト指向モデル

1. Object-Oriented Programming 的基本概念 [23.2] **(3)** [22.8] **(4)**

Explain **object-oriented programming** with the following keywords:**"Object", "Class", "Instance", and "Message passing".**

Object-oriented programming organizes software using **objects**.

- A **class** is a blueprint that defines data and behavior.
- An **object** is an entity that has data and performs actions.
- An **instance** is a specific object created from a class.
- **Message passing** means that objects communicate by calling each other's methods.

Example:

Car is a class, and myCar is an instance of the class.

When myCar.start() is called, a message is sent to the object to perform the start action.

---

2. Polymorphism [23.2] **(4)** [22.8] **(5)**

Explain **polymorphism** in object-oriented programming with an example.

Polymorphism is the ability to use the same method name for different types of objects, while each object performs the method in its own way.

For example, Dog and Cat are subclasses of Animal. Both have a method called speak(). When speak() is called, a dog returns "Woof," while a cat returns "Meow." Therefore, the same method call produces different behavior depending on the object.

---

3. Multiple Inheritance [23.8] **(5)**

Explain the concepts of **"Multiple inheritance"** and **"Polymorphism"** in object-oriented programming with

examples, respectively.

Multiple inheritance is a feature in object-oriented programming where one class inherits attributes and methods from more than one parent class.

For example, a FlyingCar class can inherit from both the Car class and the Aircraft class. Therefore, it can use the driving functions of Car and the flying functions of Aircraft.

第 7 回　プロジェクト管理

1. Project 的定义 [24.2] **(3)**

Explain what a **"project"** is.

A project consists of limited resources, a fixed deadline, and specific goals.

2. 大规模软件开发困难的原因 [24.2] **(4)**

List and discuss **three reasons why large-scale software development is difficult**.

①The object being developed is intangible software.

Because software cannot be directly seen or touched, it is difficult for managers to judge development progress accurately.

②There is no standard software development process.

Development processes differ greatly among organizations, so there is not enough reliable knowledge and experience that can be applied to every project.

③Large-scale development projects tend to be one-time projects.

Since each project is different and technology changes rapidly, previous experience cannot always be reused to predict future problems.

第 8 回　形式手法

第 9 回　設計技法

Design Patterns [26.2] **(2-b)** [24.8] **(3)**

Explain the term **"Design Patterns"** and its importance.

① Design patterns are reusable and proven solutions to commonly occurring software design problems.

② Importance: reuse proven solution, improve software quality, improve maintainability, improve team communication, increase flexibility, reduce development time.

第 10 回　ソフトウェアモジュール

第 11 回　検証技術

1. 软件测试的四个层次 [23.8] **(2)**

About software testing, explain the following testing methods:

unit testing, integration testing, system testing, and operational testing.

①unit testing: each individual module functions correctly on its own.

②integration testing: this confirms consistency between modules, relationships with other systems,

③system testing: checks whether the whole system works correctly according to the design specification.

④operational testing: checks whether the system works correctly in the actual working environment and satisfies real user requirements.

---

2. peer-review [22.2] **(5)** [21.8] **(5)**

Explain what the **peer-review in software developments** is.

Peer review in software development means that other developers check your code or design to find problems and improve quality before it is released.

第 12 回　ソフトウェアの保守・進化

第 13 回　ソフトウェアメトリクス

Function Point Method [22.8] **(3)** [22.2] **(3)** [21.8] **(3)**

Explain the **function point method** with an example.

The function point method estimates software size by counting user-visible functions and applying a weight to each function.

Example: Assume all functions have low complexity.

Function	Number	Weight	Score
Inputs	3	3	9
Outputs	2	4	8
Inquiries	1	3	3
Internal files	2	7	14

Total: (9+8+3+14=34) function points.

The 34 function points can be used to estimate the development time, cost, and effort.

第 14 回　安全・安心な社会のためのソフトウェア

Part B　Multiagent Systems

1. Collective Intelligence

2. Multiagent Systems

[21.2] **(4)**

Explain the procedure of the **Contract Net Protocol**, and explain the **"mutual selection"** that is one of its characteristics.

3. Agents and AI [25.8] **(3)** [25.2] **(4)**

Explain **"agents"** by using the word **"environment"**.

An agent is an entity that perceives its environment and takes action in that environment to achieve specific goals.

4. Non-cooperative Games: Nash Equilibrium and Extensive Form

Complete Information Games 与 Incomplete Information Games [25.8] **(4)**

List two specific examples of well-known games that are considered **complete information games** and **incomplete information games**, for each, and explain why.

① Complete information games: prisoner's dilemma, matching pennies

Because all players know the strategy sets and payoff functions of all participants.

② Incomplete information games: first-price Auction, Bayesian Game (Poker)

Because players have private information, which is not known to other players, and must reason under uncertainty.

2. 3×3 Payoff Matrix 的 Pure-Strategy Nash Equilibrium

[21.2] **(1)**

Show the **pure-strategy Nash equilibrium** of the following **pay-off matrix** and explain how to derive it.

Player 1 \ Player 2	A	B	C
A	(4, 5)	(5, 6)	(6, 2)
B	(2, 8)	(7, 2)	(1, 3)
C	(6, 7)	(2, 2)	(4, 5)

3. 2×2 Payoff Matrix 的 Pure 与 Mixed Nash Equilibria

[21.2] **(2)**

Show all the **pure-strategy Nash equilibria** and the **mixed-strategy Nash equilibria** of the following pay-off matrix and explain how to derive them.

Player 1 \ Player 2	A	B
A	(4, 3)	(1, 1)
B	(1, 1)	(3, 4)

5. Cooperative Games: Coalition, Core, Shapley Value

6. Voting and Strategic Manipulation

7. Mechanism Design and Auctions

1. First-Price Sealed-Bid Auction

[25.2] **(5)**

Answer questions (a), (b), and (c) regarding the **internet auction**:

- One painting is being auctioned.
- Three bidders, **Alice, Bob, and Caroline**, are participating.
- Each participant's bid amount is as follows:
    - Alice's bid: **$100**
    - Bob's bid: **$80**
    - Caroline's bid: **$40**

[25.2] **(5-a)**
If we use a **first-price sealed-bid auction**, who will win the painting, and how much will the winner pay?

---

2. English Auction
[25.8] **(5)**
On an internet auction site, four bidders want to buy a rare painting. The **"true value"** (the highest amount they are willing to pay) of the painting for each bidder is as follows.

Bidder	True value
A	2,000,000 yen
B	2,600,000 yen
C	2,400,000 yen
D	2,200,000 yen

When using the **English auction**, where bidders bid rationally, answer the following questions.
Let the minimum bid increment be **α (>0) yen**.
[25.8] **(5-1)**
Which bidder will win the auction?
[25.8] **(5-2)**
What is the winning price? Explain the reason also.
[25.8] **(5-3)**
Describe the rational strategy of each bidder. Explain the reason also.

---

8. Vickrey Auction and VCG: Incentive Compatibility
1. Vickrey Auction 的 Winner 与 Payment
[25.2] **(5-b)**
If we use a **Vickrey auction**, who will win the painting, and how much will the winner pay?
[25.8] **(5-4)**
Describe the rational strategy of each bidder. Explain the reason as well.

---

3. Vickrey Auction 的 Truthfulness 证明
[21.2] **(3)**
Prove the **truthfulness (the dominance of truthful bidding)** of the **Vickrey auction**.
[25.2] **(5-c)**
The Vickrey auction is referred to as a **truthful (strategy-proof) auction**, meaning that bidding one's true valuation is the best strategy (there is no incentive to bid a value other than one's true valuation). Below is the proof of this statement. What does **M** represent in the following proof?
Proof:
Let the true valuation of bidder i be v_i and let b_i represent his/her bid (note that $v_i = b_i$ is not always true). Player i's utility u_i is defined as:
$u_i = v_i - p$ (p is the payment).
Assume that all bidders' bids are different.
1. When bidding $b_i > v_i$:

- If $M > b_i > v_i$, player i loses.

- If $b_i > M > v_i$, player i wins the item, but the price M is greater than his/her true valuation v_i, resulting in a loss for him/her.
- If $b_i > v_i > M$, the outcome is the same whether player i bids b_i or v_i.

Therefore, there is no incentive to bid $b_i > v_i$.

2. When bidding $v_i > b_i$:

- If $M > v_i > b_i$, player i loses.
- If $v_i > M > b_i$, player i loses.
- If $v_i > b_i > M$, the outcome is the same whether player i bids b_i or v_i.

Therefore, there is no incentive to bid $v_i > b_i$.

---

2. Vickrey Auction 数值题与 Rational Strategy

Using a **Vickrey auction**, where each bidder bids based on their true valuation, answer the following questions.

[25.8] **(5-1)**

Which bidder wins the auction?

[25.8] **(5-2)**

What is the winning price?

[25.8] **(5-3)**

If the winning bidder offers **3,000,000 yen**, what is the winning bid amount?

Explain your reasoning.